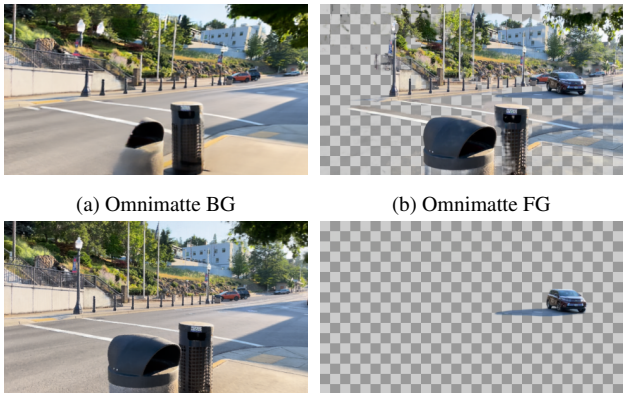
This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

OmnimatteRF: Robust Omnimatte with 3D Background Modeling

Geng Lin[1] Chen Gao[2] Jia-Bin Huang[1,2]
Changil Kim[2] Yipeng Wang[2] Matthias Zwicker[1] Ayush Saraf[2]
[1]University of Maryland, College Park [2]Meta
https://omnimatte-rf.github.io

Abstract

Video matting has broad applications, from adding interesting effects to casually captured movies to assisting video production professionals. Matting with associated effects such as shadows and reflections has also attracted increasing research activity, and methods like Omnimatte have been proposed to separate dynamic foreground objects of interest into their own layers. However, prior works represent video backgrounds as 2D image layers, limiting their capacity to express more complicated scenes, thus hindering application to real-world videos. In this paper, we propose a novel video matting method, OmnimatteRF, that combines dynamic 2D foreground layers and a 3D background model. The 2D layers preserve the details of the subjects, while the 3D background robustly reconstructs scenes in real-world videos. Extensive experiments demonstrate that our method reconstructs scenes with better quality on various videos.



(a) Omnimatte BG (b) Omnimatte FG

(c) Our BG (d) Our FG

Figure 1. **Video with parallax effects.** Limited by their 2D image representation (a), previous works such as Omnimatte fail to handle videos with parallax effects in the background. Their foreground layer (b) has to capture (dis)occlusion effects to minimize the reconstruction loss. In contrast, our method employs a 3D background (c), enabling us to obtain clean foreground layers (d).

1. Introduction

Video matting is the problem of separating a video into multiple layers with associated alpha mattes such that the layers are composited back to the original video. It has a wide variety of applications in video editing as it allows for substituting layers or processing them individually before compositing back, and thus has been studied well over decades. In typical applications like rotoscoping in video production and background blurring in online meetings, the goal is to obtain the masks containing only the object of interest. In many cases, however, it is often preferred to be able to create video mattes that include not only the object of interest but also its associated effects, like shadow and reflections. This could reduce the often-required, additional manual segmentation of secondary effects and help increase realism in the resulting edited video. Being able to factor out the related effects of foreground objects also helps reconstruct a clean background, which is preferred in applications like object removal. Despite these benefits, this problem is much more ill-posed and has been much less explored than the conventional matting problem.

The most promising attempt to tackle this problem is Omnimatte [21]. *Omnimattes* are RGBA layers that capture dynamic foreground objects and their associated effects. Given a video and one or more coarse mask videos, each corresponding to a foreground object of interest, the method reconstructs an *omnimatte* for each object, in addition to a static background that is free from all of the objects of interest *and* their associated effects. While Omnimatte [21] works well for many videos, it is limited by its use of homography to model backgrounds, which requires the background be planar or the video contains only rotational motion. This is not the case as long as there exists parallax caused by camera motions and objects occlude each other. This limitation hinders its application in many real-world videos, as shown in Fig. 1.

D^{2}NeRF [36] attempts to address this issue using two

radiance fields, which model the dynamic and static part of the scene. The method works entirely in 3D and can handle complicated scenes with significant camera motion. It is also self-supervised in the sense that no mask input is necessary. However, it separates all *moving* objects from a static background and it is not clear how to incorporate 2D guidance defined on video such as rough masks. Further, it cannot independently model multiple foreground objects. A simple solution of modeling each foreground object with a separate radiance field could lead to excessive training time, yet it is not clear how motions could be separated meaningfully in each radiance field.

We propose a method that has the benefit of both by combining 2D foreground layers with a 3D background model. The lightweight 2D foreground layers can represent multiple object layers, including complicated objects, motions, and effects that may be challenging to be modeled in 3D. At the same time, modeling background in 3D enables handling background of complex geometry and non-rotational camera motions, allowing for processing a broader set of videos than 2D methods. We call this method *OmnimatteRF* and show in experiments that it works robustly on various videos without per-video parameter tuning. To quantitatively evaluate the background separation of a 3D scene, D^2NeRF released a dataset of 5 videos rendered with Kubrics, which are simple indoor scenes with few pieces of furniture and some moving objects that cast solid shadows. We also render five videos from open-source Blender movies [6] with sophisticated motions and lighting conditions for more realistic and challenging settings. Our method outperforms prior works in both datasets, and we release the videos to facilitate future research.

In summary, our contributions include the following:

1. We propose a novel method to make Omnimatte [21] more robust by better modeling the static background in 3D using radiance fields [22].
2. Utilizing the *omnimatte* masks, we propose a simple yet effective re-training step to obtain a clean static 3D reconstruction from videos with moving subjects.
3. We release a new dataset of 5 challenging video sequences rendered from open-source blender movies [6] with ground truths to better facilitate the development and evaluation of the video matting with associated effects (aka *omnimatting* [21]) problem.

2. Related Work

Video Matting. There is a long line of work exploring video matting due to its importance in video editing. Green screening and rotoscoping are critical first steps in any visual effects pipeline. The matting problem aims to extract the foreground subjects into their own RGBA layers and separate them from the background RGB layer, which is a highly under-constrained problem. Many approaches have utilized motion and depth cues in addition to integrating user interactions [7, 3, 32, 16, 9]. Background Video Matting [18] specifically addresses real-time video matting of people and preserving strand-level hair details.

Matting with Associated Effects. Video matting is often insufficient, as foreground subjects might have associated effects like shadows or reflections that need to be extracted into the foreground RGBA layers. This problem has not been explored as extensively and, in practice, is often dealt with manually using advanced interactive rotoscoping tools [15]. Omnimatte [21] was the first to propose a generic framework capable of learning any associated effect. Previous works often specifically addressed associated effects like shadows [34, 33]. The ability to obtain matte layers with associated effects has many exciting applications, such as re-timing motions of different people [20], consistent background editing [13, 14], background subtraction, green screening, and many other video effects [21]. Recently, FactorMatte [12] has been proposed to improve the quality with data augmentation and conditional priors. These works have in common that they take predefined masks that hint at the foreground objects and decompose each video into several layers, with one object in each layer with its associated effects. Then, there is a background layer, a 2D static image, or a deformable atlas shared by all the frames. The background is warped and cropped via a homography to render each frame. While the foreground layers have shown great potential in capturing dynamics, their single image background limits the application of these methods to videos with planar environments without parallax effects caused by camera motion.

Radiance Fields. Radiance fields (RF) emerged as 3D representations capable of capturing geometric details and photorealistic appearances [22]. Radiance fields model the 3D scene as a continuous function that maps the position and the viewing direction of any point in world space to its color and opacity. Novel views can be synthesized via volume rendering along rays cast. This continuous function is learned by optimizing with a reconstruction loss on the rendered images. This view-dependent volumetric representation can model various challenging scenes that previous surface-based methods struggled to handle: e.g., shiny surfaces like metals or fuzzy surfaces like hair or fur. Since then, it has been extended along multiple axes: better appearance modeling (e.g., reflection and refraction [31, 5, 2, 1], faster optimization [8, 27, 23] and modeling dynamic scenes [38, 17, 10, 19]. Since the MLP-based implicit RF representations are slow to train, we use voxel-based explicit radiance field representations [8] [27].

Specifically, we use the factorized voxel grid representation from [8].

Self-Supervised Video Dynamics Factoring. Another related work is video dynamics factoring without needing a predefined mask. One recent work is deformable sprites [39] that rely only on motion cues. Similar to other video matting works, it has a 2D foreground and background layers and the same limitations as Omnimatte. For modeling in 3D, D^{2}NeRF[36] proposes to decouple the scene with two radiance fields, one for the dynamic content and the other for the statics. D^{2}NeRF[36] handles a special case of matting with only one foreground object, and, compared to the other methods, it is not limited to planar backgrounds. However, the self-supervised method relies on the heuristics that require per-video hyper-parameter tuning and does not robustly generalize to new videos. The quality of the foreground reconstruction can also be limited for objects that have large nonrigid motions.

We, therefore, propose a method for video matting with associated effects that has the advantages of supervised 2D mattes that support multiple individual objects with great details, as well as 3D background decoupling that works with non-planar videos.

3. Method

The concept of *omnimattes* is proposed by Lu et al. [21], extending RGBA video mattes to capture associated effects of the objects of interest like shadows and reflections. To avoid any confusion, in the following text, we refer to their work as capital Omnimatte, and the resulting RGBA layers as italic *omnimatte*. In the matting setup, the user prepares a video of T frames $\{I_t\}_{t=1}^{T}$, and N *ordered* mask layers $\{M_t^i\}_{i=1}^{N}$, each containing a coarse mask video of an object of interest. The video's camera parameters are also precomputed as $\{P_t\}$.

The goal is to predict RGBA foreground layers C_t^i and α_t^i that contain the objects together with their associated effects, and a background layer B_t which is clean and free from the effects cast by the foreground objects. An input frame I_t should be reconstructed by alpha compositing the foreground layers above the background.

In Omnimatte, the background is represented by a static 2D image and a homography transform P_t. To compose a frame, part of the static background is extracted according to the estimated homography P_t. The key idea of our work is to represent the static background in 3D using a radiance field, while keeping the foreground in 2D to better capture the dynamics of objects. We employ an explicit factorized voxel-based radiance field [8] to model the background. In this case, P_t represents a camera pose, and a background frame is rendered with volume rendering. Note that the foreground layers are still 2D videos. We refer to this combination as the OmnimatteRF model.

3.1. The OmnimatteRF Model

An outline of our model is depicted in Figure 2. The model has two independent branches: foreground and background. For any given frame, the foreground branch predicts an RGBA image (*omnimatte*) for each object, and the background branch renders a single RGB image.

Preprocessing. Following similar works, we use an off-the-shelf model RAFT [29] to predict optical flow between neighboring frames. The flow is used as an auxiliary input and ground truth for supervision, denoted by $\{F_t\}$. We also use an off-the-shelf depth estimator MiDaS [26] to predict monocular depth maps $\{D_t\}$ for each frame and use them as ground truth for the monocular depth loss.

Background. The background branch consists of a static neural radiance field, f_{bg}, encoding the 3D representation of the scene. To render a pixel in a frame I_t, a ray is traced according to the estimated camera pose P_t, and the final RGB color is produced via volumetric rendering. The result of rendering the entire frame is $(B_t, \hat{D}_t) = f_{\text{bg}}(P_t)$, where B_t is an RGB image and $\hat{D}_t$ is a depth map.

Foreground. The foreground branch is a UNet-style convolutional neural network, f_{fg}, similar to that of Omnimatte. The input of the network is a concatenation of three maps:

1. The coarse mask M_t^i. The mask is provided by the user, outlining the object of interest. Mask values are ones if the pixels are inside the object.
2. The optical flow F_t. It provides the network with motion hints. Note that the network also predicts an optical flow as an auxiliary task (detailed in Sec. 3.2.2).
3. The feature map E_t. Each pixel (x, y) in the feature map is the positional encoding of the 3-tuple (x, y, t).

Multiple foreground layers are processed individually. For the i-th layer, the network predicts the *omnimatte* layer (C_t^i, α_t^i) and the flow $\hat{F}_t^i$.

Detail Transfer. For a tradeoff between image quality and training time, the foreground network typically produces a color layer with missing details when the alpha layers have captured sufficient associated effects. To boost the output quality, Omnimatte transfers details from input frames. We include the same process in our pipeline. Note that this is a post-processing step to produce final results, and does not apply to model optimization.

3.2. Optimizing the Model

We optimize an OmnimatteRF model for every video since both branches of our model are video-specific. To supervise learning, we employ an image reconstruction loss and several regularization losses.

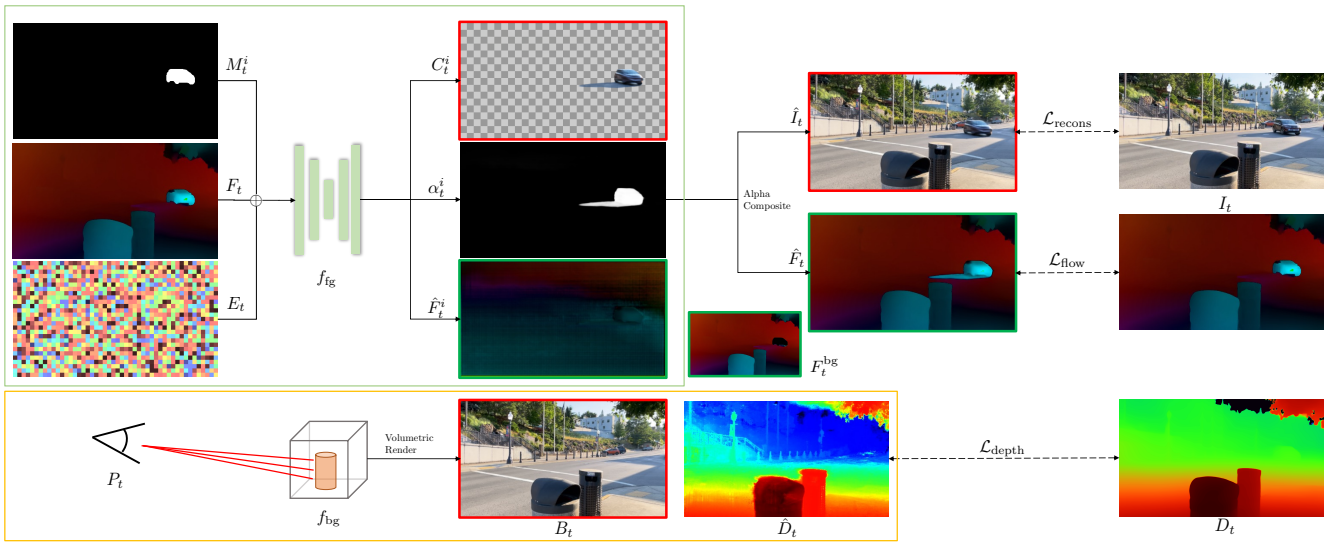


Figure 2. **Method overview.** We propose a video matting method, named OmnimatteRF, which combines 2D foreground layers with a 3D background layer. The foreground branch (f_{fg}, in green box) predicts an RGBA layer (C_t^i, α_t^i) for each object, and an auxiliary flow output ($\hat{F}_t^i$). The background branch (f_{bg}, in yellow box) produces a background layer with depths ($B_t, \hat{D}_t$). **Optimization.** During training, predicted colors ($\hat{I}_t$) and flow ($\hat{F}_t$) are alpha-composited, whose inputs have red and green borders respectively. The right most column illustrates the data terms in the loss function, and we omit the regularization terms in this illustration.

3.2.1 Reconstruction Loss

We compute the reconstruction loss with the composed image $\hat{I}_t$ by alpha composition of foreground and background layers:

$$\hat{I}_t = \sum_{i=1}^{N} \left(\prod_{j=1}^{i-1} (1 - \alpha_t^j) \alpha_t^i C_t^i \right) + \prod_{i=1}^{N} (1 - \alpha_t^i) B_t \quad (1)$$

And the reconstruction loss is the mean-squared-error between the predicted and input frame,

$$\mathcal{L}_{\text{recons}} = ||\hat{I}_t - I_t||^2 \quad (2)$$

The reconstruction loss supervises both branches of our pipeline simultaneously. Limited by the computational cost of volumetric rendering, the background layer is rendered only at sparse random locations at each step, where $\mathcal{L}_{\text{recons}}$ is computed for the composed pixel values.

3.2.2 Foreground Losses

We follow Omnimatte and include the alpha regularization loss $\mathcal{L}_{\alpha\text{-reg}}$, alpha warp loss $\mathcal{L}_{\alpha\text{-warp}}$, and flow reconstruction loss $\mathcal{L}_{\text{flow}}$. We also bootstrap the initial alpha prediction to match the input mask with the mask loss $\mathcal{L}_{\text{mask}}$, which is gradually decayed and disabled once its value drops below the threshold.

While most regularization terms in Omnimatte can be applied directly to our pipeline, the flow reconstruction loss is an exception. The formulation of the loss remains identical: given the per-layer flow prediction $\hat{F}_t^i$ and a background layer flows F_t^{bg}, the complete flow $\hat{F}_t$ is composed via alpha composition (Eq. 1). Then, the loss is defined as:

$$\mathcal{L}_{\text{flow}} = ||(\hat{F}_t - F_t) \otimes M_t^{\text{fg}}||^2 \quad (3)$$

Here, M_t^{fg} is the union of all foreground masks ($\{M_t^i\}$) for the frame I_t, and the loss is only evaluated at the location of *input* coarse masks. The authors of Omnimatte have shown the effectiveness of this loss in their case, and we also demonstrate its importance in an ablation study.

However, it remains unclear how F_t^{bg} can be obtained. In Omnimatte, the background flow can be derived from image homography, which serves both as an input to the network and a background for composition. On the other hand, since our 3D background has only known camera poses but not depths, we cannot obtain background flows directly. Instead, we use the ground truth flow F_t as network input to provide motion cues and a masked version of F_t as background flow for composition. The masked flow is $F_t^{\text{m}} = F_t \otimes (1 - M_t^{\text{fg}})$, which is the ground truth optical flow with the regions marked in the coarse masks set to zeros. $\otimes$ denotes elementwise multiplication. We find it crucial to use F_t^{m} rather than F_t for composition, as the latter case encourages the network to produce empty layers with α_t^i equal to zero everywhere.



(a) (b) (c)

Figure 3. **Background Layer Training Signals.** We illustrate how the training signal to the background layer changes over time. It explains why the background captures some of the associated effects (in this example, shadows). We use the pixel circled in red as an example. (a) At the beginning of training, the foreground alpha value (in light green) does not include the shadow. Therefore, α is small and at this pixel, $\hat{I}_t(x,y) \approx B_t(x,y)$. The reconstruction loss $\mathcal{L}_{\text{recons}}$ encourages the background network f_{bg} to produce dark prediction at this location from this viewing angle. (b) As training progresses, α gets larger in the shadow region, and $\hat{I}_t(x,y) \approx C_t^i(x,y)$. This means that f_{bg} receives little to no supervision signals from this pixel. If it has modeled the shadow in some ways (in this case, a hole), it has little incentive to remove it, leaving the artifact in (c).

3.2.3 Background Losses

Apart from the reconstruction loss, the background network is supervised by the total variation regularization loss, $\mathcal{L}_{\text{bg-reg}}$, as in TensoRF [8]. In addition, monocular depth supervision is used to improve scene reconstruction when the camera motions consist of rotation only:

$$\mathcal{L}_{\text{depth}} = \text{metric}(D_t, \hat{D}_t), \tag{4}$$

where $\hat{D}_t$ is the estimated depth from volume rendering [22], and the metric function is the scale-invariant loss from MiDaS [26]. Also, we empirically find that $\mathcal{L}_{\text{depth}}$ can introduce floaters, and employ the distortion loss $\mathcal{L}_{\text{distort}}$ proposed in Mip-NeRF 360 [4] to reduce artifacts in the background.

3.2.4 Summary

The combined loss for joint optimization is:

$$\begin{aligned}\mathcal{L} = &\mathcal{L}_{\text{recons}} + \underbrace{\mathcal{L}_{\alpha\text{-reg}} + \mathcal{L}_{\alpha\text{-warp}} + \mathcal{L}_{\text{flow}} + \mathcal{L}_{\text{mask}}}_{\text{Foreground}} + \\ &\underbrace{\mathcal{L}_{\text{bg-reg}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{distort}}}_{\text{Background}}\end{aligned} \tag{5}$$

At every optimization step, $\mathcal{L}_{\text{recons}}$ and background losses are evaluated at sparse random locations. Foreground losses are computed for the full image.

3.3. Clean Background via Masked Retraining

When the pipeline is trained jointly as described above, it is sometimes observed that the background radiance field models some of the foreground contents like shadows (see Fig. 3(c)). Compared to 2D images, 3D radiance fields are so much more capable that they can exploit distorted geometry constructs, such as holes and floaters, to capture some temporal effects, although the models are given no time information. For example, as the camera moves over time, there may be a correlation between whether a surface is covered by shadow and the direction the surface is viewed from.

We illustrate this problem in Fig. 3 and explain the cause at an intuitive level. The foreground branch is bootstrapped to produce alpha values that match the coarse mask inputs, which include only the object without the associated effects. In other words, α_t values are close to one at the object, but zero in the shadows (for simplicity, we consider one foreground layer in which the object casts a shadow, like in Fig. 3). At a pixel (x,y) covered by shadow, Eq. 1 simply collapses to $\hat{I}_t(x,y) \approx B_t(x,y)$. The reconstruction loss will therefore encourage $B_t(x,y)$ to match the color of the shadow for a ray shot toward this location.

As training proceeds, f_{fg} will then gradually increase the predicted alpha values at the shadowed regions. If the shadow is hard and α gets close to one, Eq. 1 evaluates to $\hat{I}_t(x,y) \approx C_t^i(x,y)$, and the reconstruction loss gives little to no constraint to the background color at the pixel. As a result, f_{bg} is unable to learn to remove the shadow color that it produces for the ray towards frame I_t at (x,y).

There are also cases where the shadow is soft and α is in between. In these cases, the problem remains ambiguous.

Therefore, we propose to obtain clean background reconstruction via an optional optimization step. In joint training, the foreground *omnimatte* layers can capture most associated effects, including the parts with leaked content in the background layer. The alpha layers α_t can then be used to train a radiance field model from scratch, with no samples from the foreground region where alpha values are high. We show in the ablation study (see Fig. 7) that this step produces cleaner background reconstruction for in-the-wild videos. As only the background is optimized, the process is fast and takes less than an hour to complete.

4. Evaluation

We compare our quantitative and qualitative methods with Omnimatte and D^{2}NeRF [21, 36], which are state-of-the-art methods in 2D video matting and 3D video segmentation, respectively. In addition, we compare with Layered Neural Atlas (LNA) [13], which uses a deformable 2D background in contrast to Omnimatte's static image.

4.1. The Movies Dataset

Quantitative evaluation of background segmentation requires a dataset with both input videos and ground-truth background imagery. Prior works primarily use datasets like CDW-2014 [35], which are limited to mostly static backgrounds and are not applicable to our settings. Recently, `Kubrics` is proposed in D^{2}NeRF, which enables the evaluation of 3D background synthesis. However, these videos have relatively simple scenes and lighting. To facilitate the evaluation of video matting and background segmentation in challenging scenarios, we select six clips from three Blender movies in Blender Studio [6]. Compared to `Kubrics`, they feature more complicated scenes and lighting conditions, large nonrigid motion of the characters, and higher resolution. To ensure usability, we manually edit the camera trajectories so that there are sufficient camera motions and the actors have reasonable sizes. We render the clips with and without the actors to obtain input and ground truth for background reconstruction evaluation purposes. The camera poses are also exported.

4.2. Experiment Setup

We evaluate the performance of our proposed method on four datasets.

1. `Movies`: our novel challenging dataset.
2. `Kubrics`: the dataset generated and used in D^{2}NeRF, which consists of five scenes of moving objects from 3D Warehouse [30] rendered with Kubric [11].
3. `DAVIS` [24, 25]: short clips with moving foreground subjects, like humans, cars, and animals. This dataset is widely used to evaluate 2D-background matting methods [21, 13, 39].
4. `Wild`: in-the-wild sequences collected from the internet that are closer to casually captured videos, with natural and noisier camera motions, including translations and rotations, as well as objects at different distances from the camera. Naturally, these videos have backgrounds that are challenging for pure 2D methods.

`Kubrics` and `Movies` are synthetic datasets with clean background layer renderings available. Note that novel view synthesis is not the focus of our method, so we evaluate the background with input views. Both datasets have known camera poses and object masks which are used for training and evaluation.

`DAVIS` and `Wild` are real-world videos without clean background. Therefore, we only perform a qualitative evaluation to demonstrate the robustness of our method. For videos in `Wild` we recover camera poses with COLMAP. For videos that COLMAP cannot process reliably, including `DAVIS` videos, we use poses from RoDynRF [19].

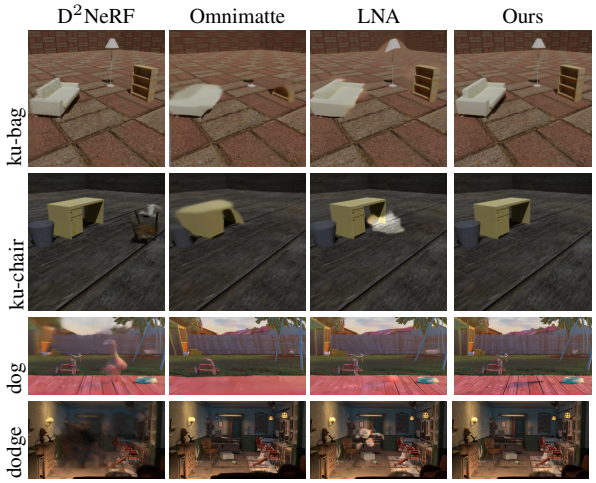


Figure 4. **Background Reconstruction.** We show examples of results presented in quantitative evaluations. For videos with parallax effects, 3D methods like D^{2}NeRF and ours reconstruct less distorted background than Omnimatte and LNA.

To obtain coarse object masks, we attempt to extract them with pre-trained object segmentation models from Detectron 2 [37]. In case it does not work, we use the Roto Brush tool in Adobe After Effects. Detailed procedures are described in the supplementary material. It takes about 10 minutes of manual effort to produce a 200-frame mask.

For all videos, we also estimate homographies with LoFTR [28] and OpenCV to enable Omnimatte processing.

As mentioned in D^{2}NeRF [36], the method is sensitive to hyperparameters. The authors released five sets of configurations for different videos. We experiment with every video using all provided configurations and report the best-performing ones.

4.3. Implementation Details

Our network is built upon the publicly available official implementation of Omnimatte [21], and TensoRF [8]. The videos in `Kubrics` have resolution 512×512, and all methods run at the resolution 256×256. For videos in other datasets with a higher resolution of 1920×1080, we downsample them by a factor of 4.

We optimize the networks for up to 15,000 steps. The learning rate of f_{fg} is set to 0.001 and is exponentially decayed after 10,000 steps. For f_{bg} we use the learning rate scheduling scheme of TensoRF. Training takes up to 6 hours on a single RTX3090 GPU. Detailed network architecture, hyper-parameters and timing data are presented in the supplementary. Our code and datasets will also be made publicly available.

4.4. Quantitative Evaluation

We quantitatively evaluate the background reconstruction quality of our method on two synthetic datasets. We

Kubrics	Car			Cars			Bag			Chair			Pillow		
	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑
D^{2}NeRF	0.135	0.854	34.10	0.105	0.859	34.77	0.131	0.880	33.98	0.090	0.916	33.29	0.105	0.926	38.80
Omnimatte	0.162	0.819	31.14	0.157	0.834	31.20	0.271	0.796	23.64	0.175	0.865	26.91	0.270	0.841	21.17
LNA	-	-	-	-	-	-	0.138	0.835	27.08	0.105	0.881	21.21	0.080	0.923	31.66
Ours	**0.033**	**0.958**	**39.09**	**0.032**	**0.961**	**39.78**	**0.029**	**0.972**	**39.58**	**0.023**	**0.977**	**42.46**	**0.022**	**0.982**	**43.62**
Movies	Donkey			Dog			Chicken			Rooster			Dodge		
	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑
D^{2}NeRF	-	-	-	0.370	0.694	22.73	-	-	-	0.340	0.708	25.13	0.408	0.729	20.95
Omnimatte	0.315	0.653	19.11	0.279	0.706	21.74	0.312	0.704	20.95	0.220	0.741	23.14	0.067	0.879	23.88
LNA	0.104	0.849	18.79	0.154	0.828	26.08	0.190	0.818	19.22	0.131	0.804	26.46	0.068	0.937	24.94
Ours	**0.005**	**0.990**	**38.24**	**0.030**	**0.976**	**31.44**	**0.021**	**0.978**	**32.86**	**0.024**	**0.969**	**27.65**	**0.006**	**0.991**	**39.11**

Table 1. **Quantitative evaluations.** We present the background reconstruction comparison of our method and baselines on the `Kubrics` and `Movies` datasets. Best results are in **bold** and second place are underlined. Results marked - are the ones the method failed to give good separations (visuals in supplementary).

report PSNR, SSIM and LPIPS for all videos in Table 1, and some visualizations in Fig. 4. For D^{2}NeRF, we tried every provided pre-set configuration for every video in `Movies`, and it only gave good results for the Dog, Rooster, and Dodge videos. Omnimatte and LNA with the 2D background layers struggles in both datasets. Our method can handle these videos well.

4.5. Qualitative Evaluation

We present a qualitative comparison of the methods in Fig. 5. Due to space limitations, we present at least one video from every dataset but show a frame from every selected video in the figure. The original videos are available in supplementary and we highly recommend watching them. D^{2}NeRF works well for the fine-tuned videos but not for new inputs without further hyper-parameter tuning. Omnimatte background has significant distortion around objects, and its foreground layer has to compensate for the limitation by capturing all residuals. Our method is versatile enough to perform well for a variety of videos with our 3D background model.

4.6. Ablation Studies

4.6.1 Loss Terms

We present background reconstruction results without $\mathcal{L}_{\text{depth}}$ in Fig. 6. For video sequences with rotational camera poses, the model struggles to extract 3D information from the input videos because of a lack of 3D clues. This loss is critical to extending our method to a broader range of videos. The effects of $\mathcal{L}_{\text{flow}}$ are also demonstrated in Fig. 6. The auxiliary task improves foreground quality and reduces unrelated content.

4.6.2 Clean Background Retraining

We employ an additional step for real-world sequences to optimize a clean background from scratch. In Fig. 7, we compare the background layer from the initial joint optimization and the final result. This is a simple yet robust way to obtain a better background.

4.7. Limitations

We list some limitations that future works can explore.

1. If a background region is covered by shadows nearly all of the time, the background model cannot recover its color correctly. An example from a `Movies` video is shown in Fig. 8. In theory, an *omnimatte* layer has an alpha channel and can capture only the additive shadow that allows the background to have the original color. However, this problem is largely underconstrained in the current setting, making it ambiguous and leading the background to unsatisfying solutions.
2. The foreground layer captures irrelevant content. In real-world videos, unrelated motions often exist in the background, like swaying trees and moving cars. These effects cannot be modeled by the static radiance field and will be captured by the foreground layer regardless of their association with the object. Possible directions include i) using a dummy 2D layer to catch such content or ii) a deformable 3D background model with additional regularization to address the ambiguity as both background and foreground can model motion.
3. Foreground objects may have missing parts in the *omnimatte* layers if they're occluded. Since our foreground network predicts pixel values for alpha composition, it does not always hallucinate the occluded parts.
4. The video resolution is limited. This is primarily due to the U-Net architecture of the foreground model inherited from Omnimatte. Higher resolutions can potentially be supported with the use of other lightweight image encoders.
5. The foreground layer may capture different content when the weights are randomly initialized differently. We include visual results in the supplementary materials.

5. Conclusion

We propose a method to obtain *omnimattes*, RGBA layers that include objects and their associated effects by com-

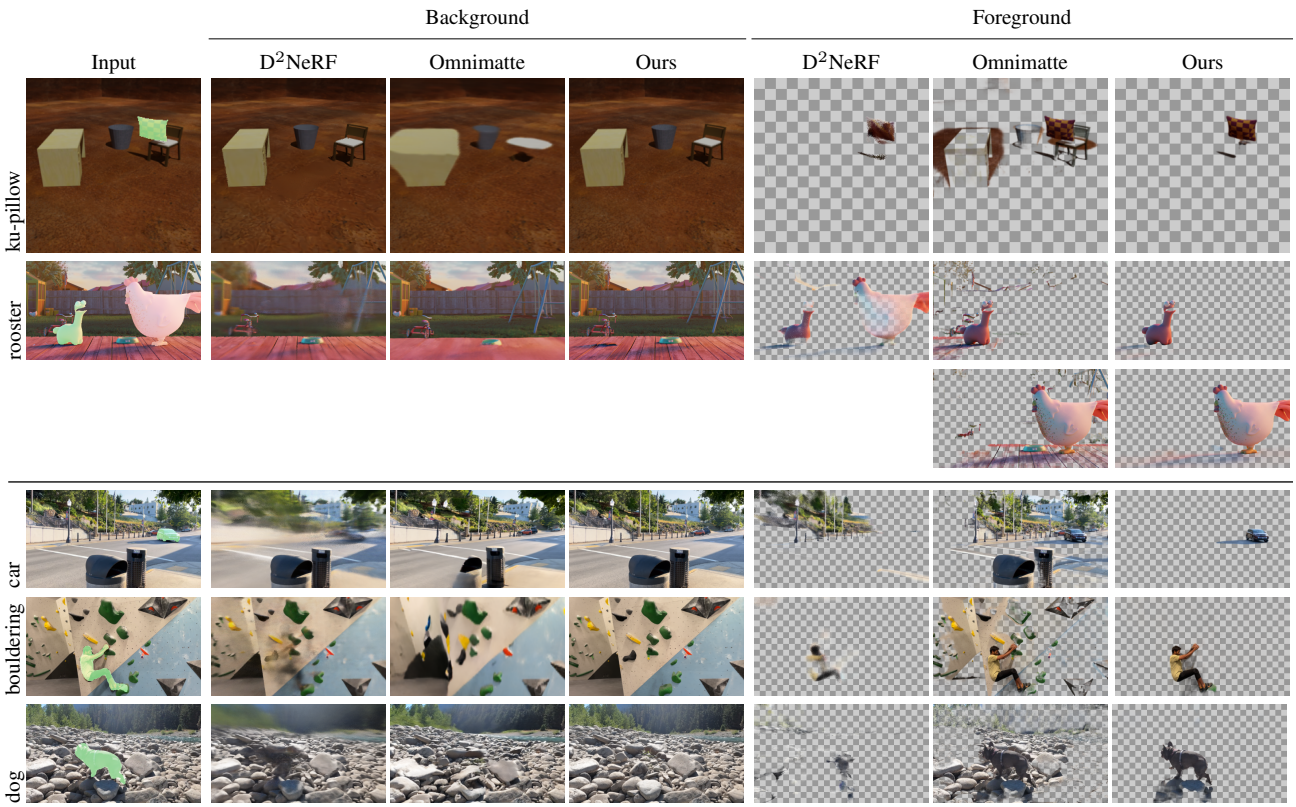


Figure 5. **Qualitative comparison.** We compare results of our and baseline methods on videos from each dataset. Readers are strongly encouraged to view videos files of more sequences available in the supplementary. The first two videos are synthetic from `Kubrics` and `Movies`, followed by three `Wild` videos. Omnimatte fails to handle objects in 3D and produces distorted background. D^{2}NeRF works for videos with appropriate hyper-parameters, but does not generalize to new videos easily. Our method handles videos in many different settings. Due to space constraint we defer LNA results to the supplementary.

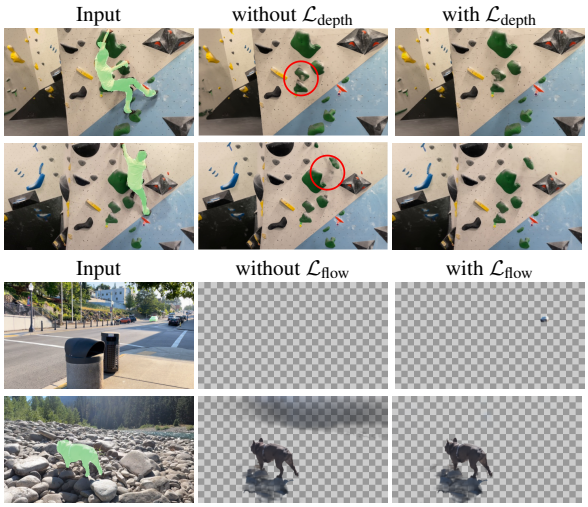


Figure 6. **Loss Term Ablations.** Background of real-world videos without $\mathcal{L}_{\text{depth}}$ and foreground without $\mathcal{L}_{\text{flow}}$ can de degraded for real-world videos.

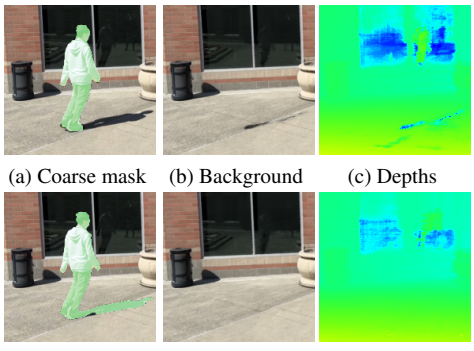
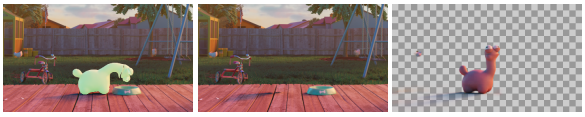


Figure 7. **Clean Background Retraining.** Background layers jointly trained can capture the shadows as a hole on the ground (a-c). After the joint training, the foreground *omnimatte* provides a better mask that can be used to train a clean background (d-f).

bining 2D foreground layers and a 3D background model. Extensive experiments demonstrate that our approach is applicable to a wide variety of videos, expanding beyond the capabilities of previous methods.



(a) (b) (c)

Figure 8. **Limitations.** (a), (b): Background can have baked-in shadows when the region is covered by shadows for most frames of the video. (c): The foreground layer captures irrelevant object motions (middle left) in the background. Best viewed in videos.

References

[1] Benjamin Attal, Jia-Bin Huang, Christian Richardt, Michael Zollhoefer, Johannes Kopf, Matthew O'Toole, and Changil Kim. Hyperreel: High-fidelity 6-dof video with ray-conditioned sampling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2

[2] Benjamin Attal, Jia-Bin Huang, Michael Zollhöfer, Johannes Kopf, and Changil Kim. Learning neural light fields with ray-space embedding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2

[3] Xue Bai, Jue Wang, David Simons, and Guillermo Sapiro. Video snapcut: robust video object cutout using localized classifiers. *ACM Trans. Graph.*, 28(3):70, 2009. 2

[4] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. *CVPR*, 2022. 5

[5] Mojtaba Bemana, Karol Myszkowski, Jeppe Revall Frisvad, Hans-Peter Seidel, and Tobias Ritschel. Eikonal fields for refractive novel-view synthesis. In Munkhtsetseg Nandigjav, Niloy J. Mitra, and Aaron Hertzmann, editors, *SIGGRAPH '22: Special Interest Group on Computer Graphics and Interactive Techniques Conference, Vancouver, BC, Canada, August 7 - 11, 2022*, pages 39:1–39:9. ACM, 2022. 2

[6] Institute Blender, Oct 2022. 2, 6

[7] Gabriel J. Brostow and Irfan A. Essa. Motion based decompositing of video. In *Proceedings of the International Conference on Computer Vision, Kerkyra, Corfu, Greece, September 20-25, 1999*, pages 8–13. IEEE Computer Society, 1999. 2

[8] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. Tensorf: Tensorial radiance fields. In *European Conference on Computer Vision (ECCV)*, 2022. 2, 3, 5, 6

[9] Yung-Yu Chuang, Aseem Agarwala, Brian Curless, David Salesin, and Richard Szeliski. Video matting of complex scenes. *ACM Trans. Graph.*, 21(3):243–248, 2002. 2

[10] Chen Gao, Ayush Saraf, Johannes Kopf, and Jia-Bin Huang. Dynamic view synthesis from dynamic monocular video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021. 2

[11] Klaus Greff, Francois Belletti, Lucas Beyer, Carl Doersch, Yilun Du, Daniel Duckworth, David J Fleet, Dan Gnanapragasam, Florian Golemo, Charles Herrmann, Thomas Kipf, Abhijit Kundu, Dmitry Lagun, Issam Laradji, Hsueh-Ti (Derek) Liu, Henning Meyer, Yishu Miao, Derek Nowrouzezahrai, Cengiz Oztireli, Etienne Pot, Noha Radwan, Daniel Rebain, Sara Sabour, Mehdi S. M. Sajjadi, Matan Sela, Vincent Sitzmann, Austin Stone, Deqing Sun, Suhani Vora, Ziyu Wang, Tianhao Wu, Kwang Moo Yi, Fangcheng Zhong, and Andrea Tagliasacchi. Kubric: a scalable dataset generator. 2022. 6

[12] Zeqi Gu, Wenqi Xian, Noah Snavely, and Abe Davis. Factormatte: Redefining video matting for re-composition tasks, 2022. 2

[13] Yoni Kasten, Dolev Ofri, Oliver Wang, and Tali Dekel. Layered neural atlases for consistent video editing. *ACM Transactions on Graphics (TOG)*, 40(6):1–12, 2021. 2, 5, 6

[14] Yao-Chih Lee, Ji-Ze Genevieve Jang, Yi-Ting Chen, Elizabeth Qiu, and Jia-Bin Huang. Shape-aware text-driven layered video editing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2

[15] Wenbin Li, Fabio Viola, Jonathan Starck, Gabriel J. Brostow, and Neill D.F. Campbell. Roto++: Accelerating professional rotoscoping using shape manifolds. *ACM Transactions on Graphics (In proceeding of ACM SIGGRAPH'16)*, 35(4), 2016. 2

[16] Yin Li, Jian Sun, and Heung-Yeung Shum. Video object cut and paste. *ACM Trans. Graph.*, 24(3):595–600, 2005. 2

[17] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*, pages 6498–6508. Computer Vision Foundation / IEEE, 2021. 2

[18] Shanchuan Lin, Andrey Ryabtsev, Soumyadip Sengupta, Brian Curless, Steve Seitz, and Ira Kemelmacher-Shlizerman. Real-time high-resolution background matting. *arXiv*, pages arXiv–2012, 2020. 2

[19] Yu-Lun Liu, Chen Gao, Andreas Meuleman, Hung-Yu Tseng, Ayush Saraf, Changil Kim, Yung-Yu Chuang, Johannes Kopf, and Jia-Bin Huang. Robust dynamic radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2, 6

[20] Erika Lu, Forrester Cole, Tali Dekel, Weidi Xie, Andrew Zisserman, David Salesin, William T Freeman, and Michael Rubinstein. Layered neural rendering for retiming people in video. In *SIGGRAPH Asia*, 2020. 2

[21] Erika Lu, Forrester Cole, Tali Dekel, Andrew Zisserman, William T Freeman, and Michael Rubinstein. Omnimatte: Associating objects and their effects in video. In *CVPR*, 2021. 1, 2, 3, 5, 6

[22] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020. 2, 5

[23] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.*, 41(4):102:1–102:15, 2022. 2

[24] F. Perazzi, J. Pont-Tuset, B. McWilliams, L. Van Gool, M. Gross, and A. Sorkine-Hornung. A benchmark dataset and

evaluation methodology for video object segmentation. In *Computer Vision and Pattern Recognition*, 2016. 6

[25] Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alexander Sorkine-Hornung, and Luc Van Gool. The 2017 davis challenge on video object segmentation. *arXiv:1704.00675*, 2017. 6

[26] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3), 2022. 3, 5

[27] Cheng Sun, Min Sun, and Hwann-Tzong Chen. Direct voxel grid optimization: Super-fast convergence for radiance fields reconstruction. In *CVPR*, 2022. 2

[28] Jiaming Sun, Zehong Shen, Yuang Wang, Hujun Bao, and Xiaowei Zhou. LoFTR: Detector-free local feature matching with transformers. *CVPR*, 2021. 6

[29] Zachary Teed and Jia Deng. RAFT: recurrent all-pairs field transforms for optical flow. In Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm, editors, *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part II*, volume 12347 of *Lecture Notes in Computer Science*, pages 402–419. Springer, 2020. 3

[30] Inc Trimble, Oct 2022. 6

[31] Dor Verbin, Peter Hedman, Ben Mildenhall, Todd E. Zickler, Jonathan T. Barron, and Pratul P. Srinivasan. Ref-nerf: Structured view-dependent appearance for neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pages 5481–5490. IEEE, 2022. 2

[32] Jue Wang, Pravin Bhat, Alex Colburn, Maneesh Agrawala, and Michael F. Cohen. Interactive video cutout. *ACM Trans. Graph.*, 24(3):585–594, 2005. 2

[33] Tianyu Wang, Xiaowei Hu, Chi-Wing Fu, and Pheng-Ann Heng. Single-stage instance shadow detection with bidirectional relation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–11, June 2021. 2

[34] Tianyu Wang, Xiaowei Hu, Qiong Wang, Pheng-Ann Heng, and Chi-Wing Fu. Instance shadow detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020. 2

[35] Yi Wang, Pierre-Marc Jodoin, Fatih Porikli, Janusz Konrad, Yannick Benezeth, and Prakash Ishwar. Cdnet 2014: An expanded change detection benchmark dataset. In *2014 IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 393–400, 2014. 6

[36] Tianhao Wu, Fangcheng Zhong, Andrea Tagliasacchi, Forrester Cole, and Cengiz Oztireli. D^2nerf: Self-supervised decoupling of dynamic and static objects from a monocular video. In *Advances in Neural Information Processing Systems*, 2022. 1, 3, 5, 6

[37] Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo, and Ross Girshick. Detectron2. https://github.com/facebookresearch/detectron2, 2019. 6

[38] Wenqi Xian, Jia-Bin Huang, Johannes Kopf, and Changil Kim. Space-time neural irradiance fields for free-viewpoint video. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*, pages 9421–9431. Computer Vision Foundation / IEEE, 2021. 2

[39] Vickie Ye, Zhengqi Li, Richard Tucker, Angjoo Kanazawa, and Noah Snavely. Deformable sprites for unsupervised video decomposition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022. 3, 6